

FILMINAS COMENTADAS

PARADIGMAS DE PROGRAMACIÓN

INTRODUCCIÓN

El presente documento, toma como hilo conductor las filminas de clase e incluye notas y ejemplos que pretenden explicar sintéticamente los principales contenidos de la asignatura. Es una guía, que debe ser ampliada luego con la lectura bibliográfica, el material digital complementario propuesto, la resolución de modelos de programación propuestos, de trabajos prácticos y la indagación personal y autónoma por parte de los alumnos.

Debe considerarse que el documento no propone un abordaje lineal de los contenidos, donde cada tema abordado se inicia y termina en una sección del documento. Lo que se persigue es un abordaje de complejidad incremental. Así, en la medida que el alumno va incorporando conocimientos volvemos sobre algunos temas incorporando nueva información y estableciendo nuevas vinculaciones conceptuales con otros aspectos de cada paradigma o análisis comparativos entre paradigmas.

ESTRUCTURA DE PROGRAMAS EN JAVA – PAQUETES

Recordemos el concepto de **módulo** que existe en la programación estructurada y que estudiamos en asignaturas previas. Un módulo referencia a una colección de funciones o procedimientos que tienen ciertas características de accesibilidad entre sí y que se vinculaban lógicamente de alguna manera. Como ejemplo, podríamos mencionar el módulo de funciones que gestionan las cuentas de un cliente bancario, u otro módulo de funciones que gestione todo el manejo de fechas para el mismo sistema.

Un módulo tiene una **Interfase** y una **Implementación**. La **interfase**, es la parte pública del módulo, es decir, el conjunto de funciones que pueden ser invocadas desde otras que están fuera del módulo. En definitiva, aquellas que constituyen los servicios que el módulo da hacia fuera. La implementación consiste en todas aquellas funciones (u otras declaraciones) que sólo dan servicios dentro del módulo, es decir, a otras funciones del mismo módulo; pero que no son conocidas desde afuera.

Extrapolando esta idea a Java, tendríamos que un módulo puede ser un archivo fuente como el siguiente:

```

class A {

Estos no son exportables

Son parte de la implementación.

Por defecto tienen alcance de paquete
o de módulo. Osea, son friendly

...

}

class B {

..

}

```

```

public class C {

...

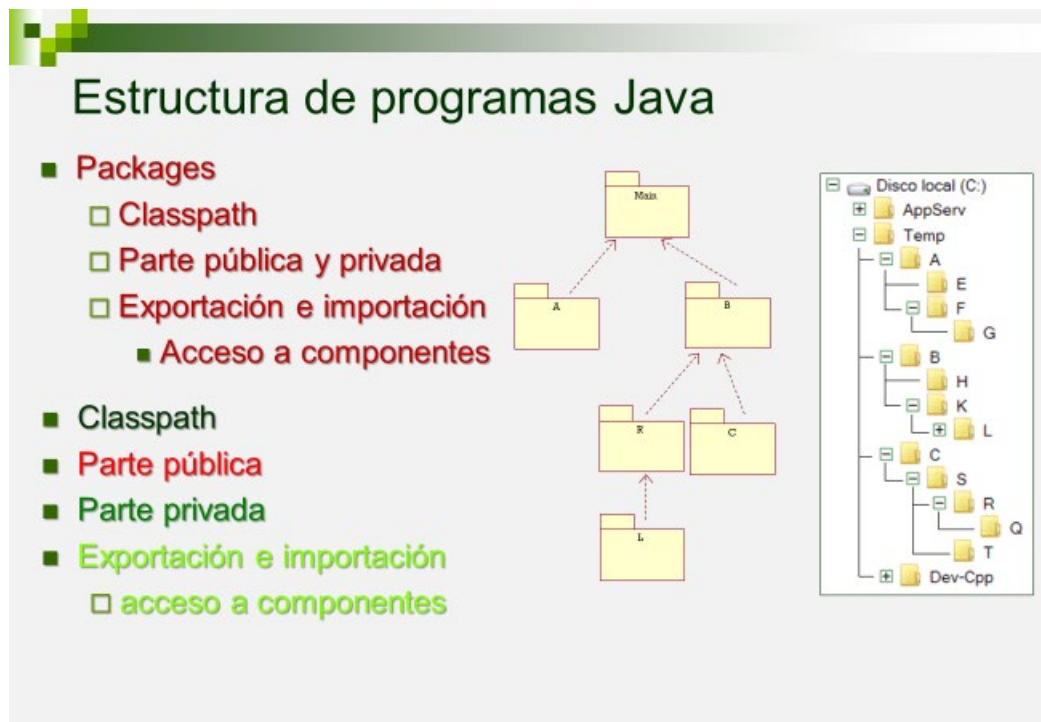
//Esta es la clase exportable

}

```

Las clases A y B, solo serán accesibles dentro del archivo, no son públicas o exportables y no se conocen afuera. Por tanto, estas clases constituirían la implementación (Parte privada) del módulo. La clase pública (o exportable) C, constituiría su interfase (Parte pública).

Más allá de esta aproximación al concepto de módulo, Java provee un mecanismo más amplio de agrupamiento de clases, que es el concepto de **Paquete** o **package**.



Un **paquete** es un conjunto de clases, lógicamente relacionadas entre sí, agrupadas bajo un nombre.

Ejemplo

El paquete `java.io` agrupa las clases que permiten a un programa realizar acciones de E/S de información. De esta forma para utilizar por ejemplo la clase `FileInputStream`, podemos hacerlo...

- llamándola por su nombre completo: `java.io.FileInputStream`
- importando previamente el paquete al que pertenece. `import java.io.FileInputStream;`

Las clases en un paquete, que son **públicas (o exportables)** constituyen la **interfase** del paquete hacia fuera o su parte pública. Aquellas que no lo son, y sólo brindan servicios dentro del propio paquete, constituyen su implementación o parte privada. Esto implica que la organización de clases en paquetes supone también, un nivel más de **protección o encapsulamiento** para las mismas. Para explicar mejor esto, recordemos cómo es la protección de clases.

Protección de clases

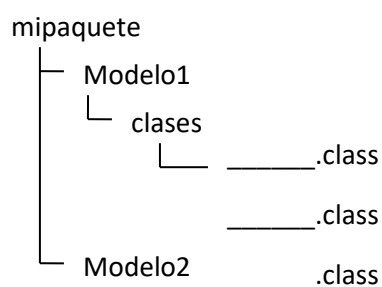
La protección de una clase determina la relación que tiene con otras clases de otros paquetes. Distinguimos **dos niveles** de protección:

- √ **de paquete**
- √ **público.**

Una clase con nivel de protección de paquete solo puede ser utilizada por las clases de su paquete. No está disponible para otros paquetes, ni siquiera para los subpaquetes (dado que un paquete puede contener otros paquetes).

Una clase pública puede ser utilizada por cualquier otra clase de otro paquete. **Por defecto, una clase, tiene el nivel de protección de paquete (friendly).**

Cuando hacemos `import mipaquete.mod1.clases.*;` lo que hace el compilador es buscar y vincular los archivos `.class` que están en la estructura de directorios que indica el nombre del paquete. Es decir, en este caso:



Estos `.class` son **clases públicas o exportables**. Bajo esta idea, cada paquete puede contener varios módulos, cada uno con una única clase pública (interfase) y otras clases de implementación (no exportables). Cuando se importa un paquete, a lo que se tiene acceso es a las clases públicas en el mismo.

En un programa X, se pueden importar y utilizar algunas o todas las clases (o módulos) incluidas en un paquete.

```
import mipaquete.modelo1.clases.*;

class Una {
    Miclase1 obj ...
}

public class PP {
    Miclase2 obj
    Miclase3 obj
}
```

Paquete Predeterminado: Toda clase Java pertenece a un paquete. Si no es uno que indicamos explícitamente, entonces por defecto, pertenece al paquete “predeterminado” o sin nombre.

En un programa Java puede aparecer cualquier número de sentencias `import`, las cuales deben escribirse antes de cualquier definición de clase. Las clases del paquete `java.lang` (que no es el predeterminado) son importadas de manera automática para todos los programas.

Resumiendo, los paquetes:

- ayudan a organizar las clases en grupos para facilitar el acceso a las mismas cuando las necesitemos en un programa. De hecho, como previamente dijimos, el nombre de un paquete tiene estricta relación con la estructura de directorios donde se alojan los archivos `.class`
- reducen los conflictos de nombres (lógicamente, la probabilidad de que dos nombres coincidan será menor mientras más elementos intervengan)
- permiten proteger las clases (una clase con nivel de protección de paquete, clase no pública, no está disponible para otros paquetes, ni siquiera para los subpaquetes).

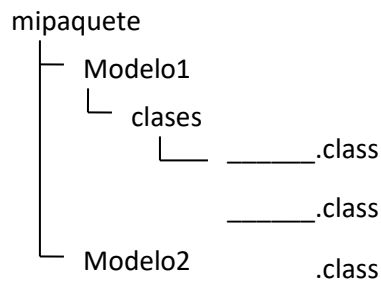
Vamos a los aspectos prácticos para operar con paquetes en Java.

Para crear un paquete

1. Elegimos el nombre que queremos darle al paquete.

Por ejemplo: `mipaquete.modelo1.clases`

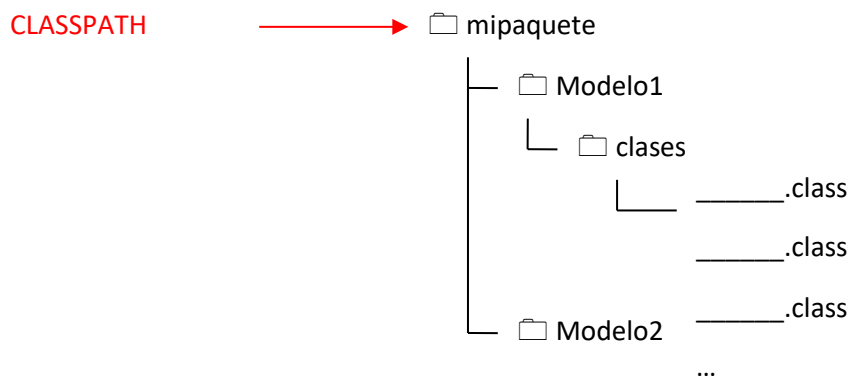
Debemos recordar que el nombre de un paquete supone una organización jerárquica para las clases. Es decir, para la estructura anterior, tendríamos que dentro del paquete mipaquete estará el subpaquete modelo1 y dentro de este el subpaquete clases. A su vez, podría haber también otros subpaquetes en la estructura.



2. Creamos la estructura jerárquica de carpetas en el disco duro.

Como vimos antes, la estructura jerárquica indicada por el nombre elegido para el paquete debe corresponder en el disco con una estructura jerárquica de carpetas, de forma que los nombres de las carpetas coincidan con los elementos de la ruta del paquete. La ruta de la carpeta raíz de esta estructura jerárquica tiene que estar especificada o apuntada por la variable de entorno **CLASSPATH**.

Para nuestro ejemplo: Deberíamos crear en el disco, la estructura de carpetas



3. Especificar el paquete al que pertenece la clase

Cuando se define una clase, se debe especificar a qué paquete pertenece utilizando la sentencia:

package nombre_paquete;

en nuestro ejemplo sería...

package mipaquete.modelo1.clases;

Esta sentencia, solo una por archivo, debe ser la primera línea del código fuente.

Nota: en la práctica real toda la estructura de clases y paquetes de una aplicación se incluye en archivos de empaquetamiento .zip o .jar. Este tipo de archivos, permite empaquetar todo lo que necesite una aplicación, incluso imágenes, archivos de audio, etc. En general, esta es la forma en que se distribuyen las aplicaciones Java, en archivos .zip, .jar o .war según sea el formato de empaquetamiento utilizado.

El uso de entornos de desarrollo simplifica las tareas de creación de paquetes y empaquetamiento de la aplicación para los programadores. El alumno deberá ver en las prácticas de laboratorio y material complementario como manejar este tema en el IDE que esté utilizando.